DESIGN AND ANALYSIS OF ALGORITHMS

(Practical 3)

SOURCE CODE

```
def fractional_knapsack(weights, values, capacity):
    n = len(weights)

    ratio = [(values[i] / weights[i], weights[i], values[i]) for i in rang
e(n)]

    ratio.sort(key=lambda x: x[0], reverse=True)

    total_value = 0.0
    fractions = [0] * n

    for i in range(n):
        if capacity == 0:
            break
        take_weight = min(ratio[i][1], capacity)
        total_value += take_weight * ratio[i][0]
        capacity -= take_weight
        fractions[i] = take_weight / ratio[i][1]

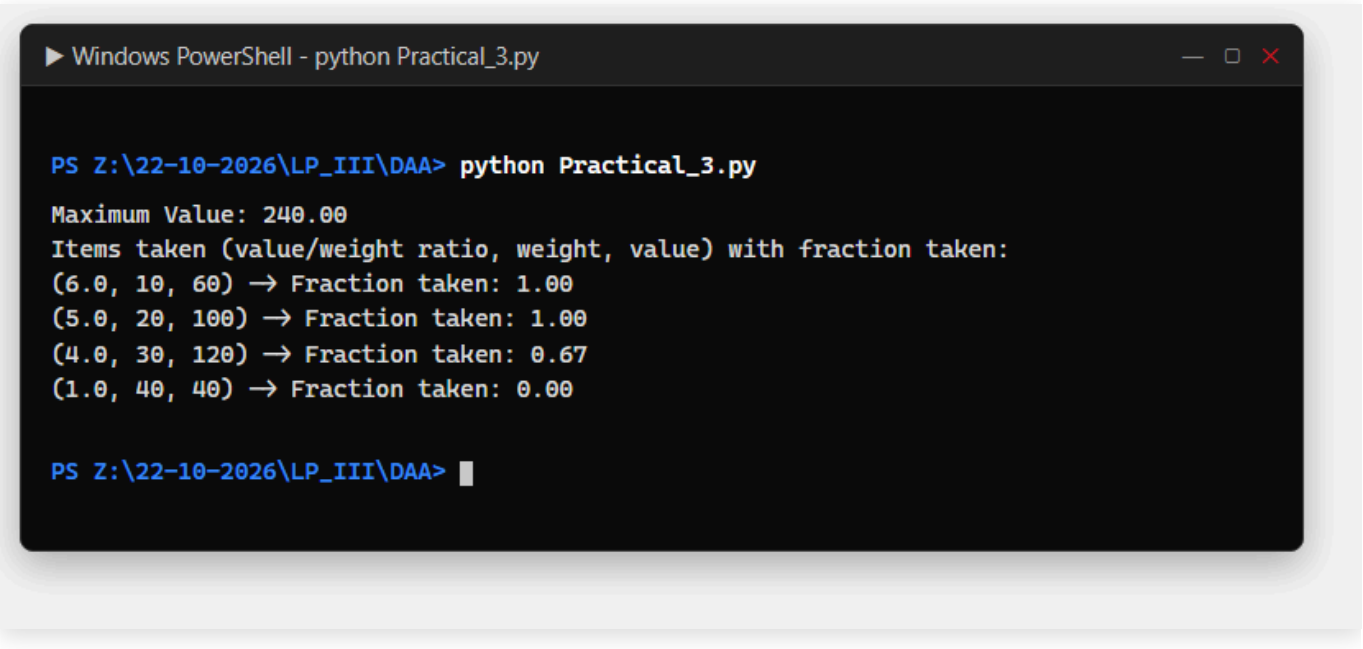
    return total_value, fractions, ratio

weights = [10, 40, 20, 30]
values = [60, 40, 100, 120]
capacity = 50

max_value, fractions_taken, sorted_items = fractional_knapsack(weights, va
lues, capacity)

print(f"Maximum Value: {max_value:.2f}")
print("Items taken (value/weight ratio, weight, value) with fraction take
n:")
for i in range(len(fractions_taken)):
    print(f"{sorted_items[i]} -> Fraction taken: {fractions_taken[i]:.2
f}")
```

OUTPUT


Windows PowerShell - python Practical_3.py
PS Z:\22-10-2026\LP_III\DAA> python Practical_3.py
Maximum Value: 240.00
Items taken (value/weight ratio, weight, value) with fraction taken:
(6.0, 10, 60) → Fraction taken: 1.00
(5.0, 20, 100) → Fraction taken: 1.00
(4.0, 30, 120) → Fraction taken: 0.67
(1.0, 40, 40) → Fraction taken: 0.00
PS Z:\22-10-2026\LP_III\DAA>